



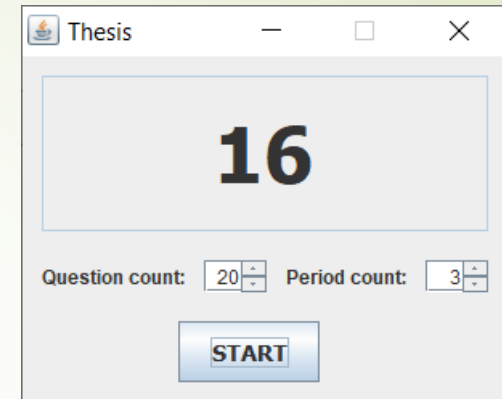
Szoftvertchnológia

Build rendszerek

Dr. Cserép Máté
ELTE Informatikai Kar
2025.

Minta alkalmazás

- Tekintsünk egy minta Java alkalmazást.
- A *ThesisGenerator* alkalmazás szóbeli vizsgákhoz képes tételszámot generálni.
 - <https://szofttech.inf.elte.hu/mate/thesisgenerator-java>
- Modell-Nézet architektúra:
 - `thesisGenerator.model` csomag:
Felhasználói felülettől független üzleti logika
 - `thesisGenerator.view` csomag:
Swing alapú felhasználói felület
 - `thesisGenerator` csomag:
Főprogram



Java programok fordítása

```
mkdir dist
javac -d dist
    src\thesisGenerator\*.java
    src\thesisGenerator\model\*.java
    src\thesisGenerator\view\*.java
```

```
cd dist
jar -cfe thesis-generator.jar
    thesisGenerator.ThesisGenerator
    thesisGenerator\*.class
    thesisGenerator\model\*.class
    thesisGenerator\view\*.class
```

```
java -jar thesis-generator.jar
```

Java programok fordítása

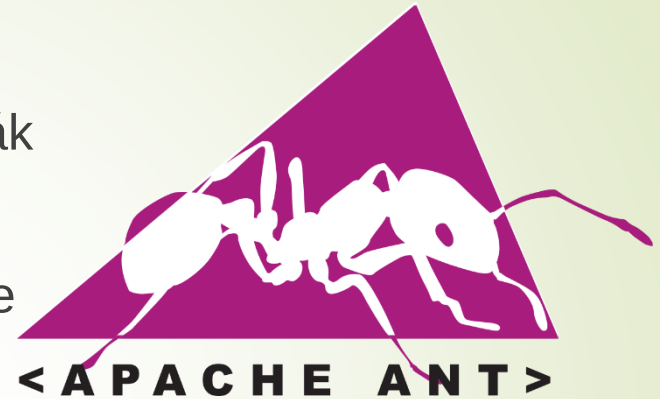
- Mint látjuk, már egy egyszerűbb, néhány forrás fájlból álló program esetén sem egyszerű a megoldás.
- **Problémák:**
 - A programok manuális fordítása már kisebb programoknál is könnyen nehezen kezelhetővé válik.
 - Nagyobb programoknál nem követhető mely fordítási egységek újrafordítása szükséges.
 - A teljes alkalmazás újrafordítása valós vállalati alkalmazásoknál hosszabb időt is igényelhet.

Elvárások a build rendszerekkel szemben

- A kód lefordítása
 - A fordítási célok (*target*) függőségeinek (*dependency*) kezelése
- A binárisok csomagolása
 - Többféle kiadási (*release*) opció támogatása
- Automatizált tesztek végrehajtása
- A binárisok kihelyezése (*deployment*) a teszt szerverre
- Kód átmásolása egyik helyről a másikra
- Csomagkezelő rendszerek (*package repository*) kezelése

Jellemzők

- Imperatív megközelítés
- Tipikusan Java projektekhez használják
- XML alapú build fájl
 - Alapértelmezetten *build.xml* a neve
- Hivatalos weboldal és tutorial:
 - <https://ant.apache.org/>
 - <https://ant.apache.org/manual/tutorial-HelloWorldWithAnt.html>
- Telepítés:
 - Windowsra a hivatalos weboldallról letölthető a telepítő.
 - UNIX rendszerekre a *package repository* tartalmazza.
 - Debian/Ubuntu: `apt-get install ant`
 - Egy IDE-vel együtt, pl. a NetBeans telepíti az Ant-ot.



Build fájl (build.xml)

- A *build.xml* egy projekt gyökérelemet tartalmaz, amely beállítja:
 - A projekt nevét
 - Az alapértelmezett célt (lásd később)
 - A projekt alapkönyvtárát, jellemzően az aktuális könyvtárat.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<project name="projname"  
        default="deftarget"  
        basedir=".">
```

```
...
```

```
</project>
```

Cél definiálása

- A projekt elemen belül többféle típusú elem definiálható, a legfontosabbak a célok (*targets*):

```
<project ...>  
  <target name="compile">  
    ...  
  </target>  
</project>
```

- A *target*-ek parancssorban is futtathatók
ant **compile**

Könyvtár létrehozása

- Cél létrehozása, amely a *classes* könyvtárat hozza létre a fordítandó *.class* fájloknak:

```
<project ...>  
  <target name="prepare">  
    <mkdir dir="classes"/>  
  </target>  
</project>
```

- Parancssor:
ant **prepare**

Célok közötti függőségek

- Definiáljunk egy célt, amely a *src* könyvtárban található összes Java forrásfájlt lefordítja, a kimenetet pedig a *classes* mappába helyezi.
 - Állítsuk be, hogy a *compile* cél függjön a *prepare* céltól!

```
<project ...>  
  <target name="compile" depends="prepare">  
    <javac srcdir="src" destdir="classes"/>  
  </target>  
</project>
```

- Parancssor:
ant **compile**

Tisztítási cél (*cleanup target*)

- Adjunk hozzá egy tisztítási célt, amely eltávolítja az összes lefordított bináris fájlt.

```
<project ...>
  <target name="clean">
    <delete>
      <fileset dir="classes" includes="*" />
    </delete>
    <delete dir="classes" />
  </target>
</project>
```

- A fájlok külön törlése nem szükséges, a classes könyvtár törlése rekurzívan eltávolítja annak tartalmát.
- Parancssor:
ant **clean**

Build rendszerek - Ant

Tisztítási cél (*cleanup target*)

- A `failonerror` attribútummal beállítható, hogy a cél sikertelensége esetén a teljes folyamat megszakadjon-e:

```
<project ...>  
  <target name="clean" failonerror="false">  
    <delete dir="classes"/>  
  </target>  
</project>
```

- Parancssor:
ant **clean**

Változók

- Egy projekten belül változókat (*properties*) is definiálhatunk.
 - A változók kulcs-érték párokat alkotnak.
 - A `${name}` szintaxis használatával futásidőben értékelődnek ki.

```
<project ...>  
  <property name="jarname"  
            value="filename.jar" />  
  
  ...  
</project>
```

- Vannak beépített változók is, például a `${basedir}` a projekt alapkönyvtára.

➤ <https://ant.apache.org/manual/properties.html>

Csomagolás

```
<project ...>
  <target name="jar" depends="compile">
    <jar destfile="${jarname}">
      <fileset dir="classes">
        <include name="*.class"/>
      </fileset>
      <manifest>
        <attribute name="Main-Class" value="Main"/>
      </manifest>
    </jar>
  </target>
</project>
```

► **Megjegyzés:** Fontos a belépési pont megadása a *manifest*-ben!

Cél: komplex példa

```
<target name="compile" depends="prepare,init">
  <javac destdir="build/classes" debug="on">
    <src path="src1/java"/>
    <src path="src2/java"/>
    <include name="**/*.java"/>
    <exclude name="com/comp/xyz/applet/*.java"/>
    <classpath>
      <fileset dir="lib">
        <include name="*.jar"/>
      </fileset>
    </classpath>
  </javac>
</target>
```

Build rendszerek - Ant

Kihelyezés (fájlműveletek)

- A végleges binárisok másolása a célhelyre:

```
<target name="install" depends="jar">
  <mkdir dir="build/war/WEB-INF/lib"/>
  <copy todir="build/war/WEB-INF/lib">
    <fileset dir="lib">
      <include name="*.jar"/>
      <exclude name="servlet-api.jar"/>
      <exclude name="catalina-ant.jar"/>
      <exclude name="el-api.jar"/>
    </fileset>
  </copy>
</target>
```

- Parancssor:
ant **install**

JVM futtatása

- Meghatározhatunk egy célt a lefordított és csomagolt JAR fájl futtatására:

```
<target name="run" depends="jar">  
  <java jar="${jarname}" fork="true" />  
</target>
```

- A `fork` attribútum biztosítja, hogy a feladat külön folyamatban és külön Java virtuális gépben (JVM) fusson.
- Parancssor:
ant **run**

JVM futtatása

► Bonyolultabb futtatási példa:

```
<target name="run">
  <java classname="com.comp.foo.TestClient"
    jvmargs="-Xdebug server=y,suspend=n">
    <classpath>
      <fileset dir="lib">
        <include name="*.jar"/>
      </fileset>
    </classpath>
  </java>
</target>
```

► Parancssor:

ant **run**

API dokumentáció generálása

```
<target name="doc">
  <tstamp>
    <format property="timestamp" pattern="d.M.yyyy"
      locale="en"/>
  </tstamp>
  <mkdir dir="doc"/>
  <javadoc sourcepath="src" destdir="doc"
    windowtitle="Projekt dokumentáció">
    <header>Nagyon fontos projekt</header>
    <footer>Javadocs fordítva ${timestamp}</footer>
  </javadoc>
</target>
```

Unit tesztek végrehajtása

```
<target name="test" depends="compile">
  <mkdir dir="report"/>
  <junit printsummary="yes" haltonfailure="no">
    <classpath location="build/classes" />
    <classpath location="build/test/classes" />
    <classpath location="lib/junit-4.12.jar" />
    <classpath location="lib/hamcrest-core-1.3.jar" />

    <formatter type="plain" />
    <test name="com.comp.foo.ServerTest"
          haltonfailure="no" todir="report" />
  </junit>
</target>
```

Unit tesztek végrehajtása

```
<target name="test" depends="compile">
  <mkdir dir="report"/>
  <junit printsummary="yes" haltonfailure="no">
    <classpath location="build/classes" />
    <classpath location="build/test/classes" />
    <classpath location="lib/junit-4.12.jar" />
    <classpath location="lib/hamcrest-core-1.3.jar" />

    <formatter type="plain" />
    <batchtest fork="yes" todir="report">
      <fileset dir="test">
        <include name="**/*Test.java" />
      </fileset>
    </batchtest>
  </junit>
</target>
```

A ThesisGenerator alkalmazás teljes build.xml fájlja

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project name="ThesisGenerator" default="jar" basedir=".">
3   <target name="clean">
4     <delete dir="build"/>
5   </target>
6   <target name="compile">
7     <mkdir dir="build/classes"/>
8     <javac srcdir="src" destdir="build/classes"/>
9   </target>
10  <target name="jar" depends="compile">
11    <mkdir dir="build/jar"/>
12    <jar destfile="build/jar/ThesisGenerator.jar" basedir="build/classes">
13      <manifest>
14        <attribute name="Main-Class" value="thesisGenerator.ThesisGenerator"/>
15      </manifest>
16    </jar>
17  </target>
18  <target name="run" depends="jar">
19    <java jar="build/jar/ThesisGenerator.jar" fork="true"/>
20  </target>
21  <target name="doc" depends="compile">
22    <tstamp>
23      <format property="timestamp" pattern="d.M.yyyy" locale="en"/>
24    </tstamp>
25    <mkdir dir="doc"/>
26    <javadoc sourcepath="src" destdir="doc" windowtitle="Thesis Generator">
27      <header>Thesis Generator</header>
28      <footer>Javadocs compiled ${timestamp}</footer>
29      <fileset dir="src/" includes="**/*.java" />
30    </javadoc>
31  </target>
32  <target name="test" depends="compile">
33    <mkdir dir="report"/>
34    <junit printsummary="yes" haltonfailure="no">
35      <classpath location="lib/junit-4.12.jar" />
36      <classpath location="lib/hamcrest-core-1.3.jar" />
37      <classpath location="build/classes" />
38      <classpath location="build/test/classes" />
39      <formatter type="xml" />
40      <formatter type="plain" />
41      <batchtest fork="yes" todir="report">
42        <fileset dir="test">
43          <include name="**/*Test.java" />
44        </fileset>
45      </batchtest>
46    </junit>
47  </target>
48 </project>
```

NetBeans

- Alapértelmezetten a NetBeans az Ant-et használja build rendszerként.
 - a *build.xml* a projekt gyökérkönyvtárában található.
 - hivatkozik a *nbproject/build-impl.xml* fájlra, amelyet a NetBeans automatikusan generál, ezt ne módosítsuk manuálisan
 - a *build.xml* fájlban definiálhatunk *hook* célokat, amelyeket a NetBeans build folyamata automatikusan meghív:
 - *-pre-init*, *-post-init*, *-pre-compile*, *-post-compile*, *-pre-jar*, *-post-jar*, *-post-clean*, stb.

Funkciók

- Szoftverprojekt-menedzsment eszköz
- Projekt fordítása, tesztek futtatása, függőségek kezelése, dokumentáció
- Csomagkezelés: függőségek automatikus letöltése
- A build folyamat deklaratív megadása
- Fix, előre definiált könyvtárstruktúra és konvenciók
- Elsősorban Java-hoz, de pluginek segítségével más programozási nyelveket is kezelhet
- XML-alapú build fájl
 - Alapértelmezett neve: *pom.xml*
- Hivatalos weboldal és egy ajánlott oktatóanyag:
 - <http://maven.apache.org/>
 - <https://www.baeldung.com/maven>



Telepítés

- Önálló telepítés
 - Binárisok letölthetők a hivatalos weboldalról
 - Egyszerűen ki kell csomagolni egy tetszőleges helyre
 - A `MAVEN_HOME` környezeti változót erre a könyvtárra kell állítani
 - A `JAVA_HOME` környezeti változónak a JDK telepítési könyvtárára kell mutatnia
 - A `MAVEN_HOME\bin` könyvtárat hozzá kell adni a `PATH` változóhoz
- Telepítés a UNIX *package repository*-ból
 - Általában elérhető
 - Debian/Ubuntu: `apt-get install maven`
- Gyakran az IDE-kkel együtt is telepítésre kerül (pl. NetBeans, IntelliJ)

Projekt

- Project Object Model (POM)
- A projekt egyedileg azonosított: a projekt csoportja (*group*), artifact azonosító (*artifact id*) és verzió (*version*) alapján. Ezt a hármat gyakran GAV rövidítéssel jelölik.
- A projekt több modulra is osztható, amelyek önállóan kezelhetők

Project Object Model (pom.xml)

- A Project Object Model (pom.xml) tartalmazza a projekt minden fontos információját:
 - azonosítók: groupId, artifactId, version
 - hogyan van build-elve a projekt
 - a build eredménye
 - tesztesetek
 - a projekt függőségei

Project Object Model (pom.xml)

- A pom.xml fájl gyökéreleme egyben `project` elem is.
- A project elemen belül kötelezően megadandó elemek:
 - `modelVersion`: a POM specifikáció verziója
 - `groupId`: a projektet létrehozó cég vagy csoport egyedi azonosítója, Java package névválasztási szabályok szerint. Ez azt jelenti, hogy fordított tartománynévvel kezdődik, pl. *hu.elte.inf*
 - `artifactId`: a projekt egyedi neve
 - `version`: a projekt verziója
 - `packaging`: a csomagolás típusa (alapértelmezett: `jar`, egyéb lehetőségek: `pom`, `maven-plugin`, `ejb`, `war`, `ear`, `rar`, `par`)

Project Object Model (pom.xml)

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mycompany.software</groupId>
  <artifactId>app</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  ...

</project>
```

Könyvtárstruktúra

- Egy Maven projekt könyvtárstruktúrája előírt konvenciókon alapul.
 - Az alapértelmezett könyvtárelrendezés felüldefiniálható projekteírókkal, de ez nem gyakori és nem javasolt.

```
my-app
|-- pom.xml
'-- src
    |-- main
    |   |-- java
    |   |   |-- com
    |   |   |   |-- mycompany
    |   |   |   |   |-- app
    |   |   |   |   |   App.java
    |   |-- resources
    |   |   META-INF
    |   |   application.properties
    |-- test
    |   |-- java
    |   |   |-- com
    |   |   |   |-- mycompany
    |   |   |   |   |-- app
    |   |   |   |   |   AppTest.java
    |-- resources
    |   test.properties
```

Könyvtárstruktúra felüldefiniálása

```
<project>
  ...
  <build>
    <directory>target</directory>
    <outputDirectory>classes</outputDirectory>
    <finalName>${project.artifactId}-${project.version}</finalName>
    <testOutputDirectory>test-classes</testOutputDirectory>
    <sourceDirectory>src/main/java</sourceDirectory>
    <scriptSourceDirectory>src/main/scripts
    </scriptSourceDirectory>
    <testSourceDirectory>/src/test/java</testSourceDirectory>
    ...
  </build>
</project>
```


Életciklus fázisok

- A Maven build rendszer egy meghatározott életciklust (*lifecycle*) követ, amely különböző fázisokból áll. A legfontosabb fázisok az alapértelmezett életciklusban:
 - **validate:** ellenőrzi, hogy a projekt helyesen van-e definiálva és minden szükséges információ elérhető-e
 - **compile:** lefordítja a projekt forráskódját
 - **test:** teszteli a lefordított kódot egy megfelelő egységtesztelési keretrendszerrel. Ezek a tesztek nem igényelhetik a kód csomagolását vagy kihelyezését
 - **package:** becsomagolja a lefordított kódot egy terjeszthető formátumba, pl. JAR
 - **integration-test:** szükség esetén feldolgozza és kihelyezi a csomagot egy tesztelési környezetbe, ahol további integrációs tesztek futtathatók

Életciklus fázisok

- **verify:** ellenőrzések lefuttatása, hogy a csomag érvényes és megfelel a minőségi kritériumoknak
- **install:** a csomag telepítése a helyi tárolóba, hogy más helyi projektek függőségként felhasználhassák
- **deploy:** integrációs vagy kiadási környezetben történik, a végső csomagot átmásolja a távoli tárolóba, hogy más fejlesztők és projektek is hozzáférhessenek
- További részletek:
<http://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html>
- Parancssor: `mvn install`
 - Az *install* fázisig hajtja végre a fázisokat az alapértelmezett életciklusban, de a *deploy* fázist nem tartalmazza.

Életciklus fázisok

- Az alapértelmezett életciklus mellett léteznek más életciklusok is, például a clean életciklus, amely a korábban lefordított binárisok eltávolítására szolgál egy projektből.
 - Ennek az életciklusnak 3 fázisa van:
pre-clean, clean, post-clean.
- Parancssor: `mvn clean install`
 - Végrehajtja a *clean*, majd az *install* fázist, valamint az összes azt megelőző fázist.
 - Végző soron ez eltávolítja és újraépíti az összes binárist.

Célok (*goals*)

- A fordítási fázisok egy vagy több célból (*goal*) állnak
- A cél egy olyan feladat, amely a projekt fordításához vagy kezeléséhez kapcsolódik
 - A célok sorrendjét a fázis kötése (*binding*) határozza meg
 - Sok fázis csak egy célt tartalmaz
 - Például a `compile` fázis a `compiler:compile` célt tartalmazza
- Nemcsak fázisokat, hanem célokat is megadhatunk a Maven parancsban, így csak az adott cél kerül végrehajtásra az előző fázisok és céljaik nélkül.
 - Például: `mvn compiler:compile`
- Egyedi fázisok és célok is definiálhatók (a `pom.xml` fájlban)

Tárolók (*repositories*)

- Egy tároló a build eredményeit (*artifacts*) és a függőségeket tartalmazza.
 - Az alapértelmezett helyi tároló a fejlesztő home könyvtárában található:
`~/.m2/repository`
- Ha egy *artifact* elérhető a helyi tárolóban, a Maven azt használja.
- Ellenkező esetben a központi tárolóból tölti le, majd eltárolja a helyi tárolóban.
 - Az első build során a hálózati forgalom és a build idő jelentősen megnőhet.
 - Az alapértelmezett központi tároló a Maven Central:
<https://repo.maven.apache.org>
 - A Maven beállítható, hogy mely tárolókat és *mirror*-okat használjon a `~/.m2/settings.xml` fájlban.
 - Vállalatok gyakran saját belső központi tárolókat üzemeltetnek.

Build folyamat eredménye

- A fordítás során létrejön a *target* könyvtár, amely a fordítás során újonnan generált fájlokat tartalmazza
 - output, pl.: *my-app-1.0-SNAPSHOT.jar*
 - *classes* könyvtár: a fordítás során létrejött osztályfájlok (*class files*), de a tesztosztályok nélkül
 - *test-classes*: teszt forrásokból készült osztályok
 - *maven-archiver/pom.properties*: a projekt GAV azonosítóját leíró fájl
 - *surefire-reports*: a tesztelési jelentések

Projekt hierarchiák

- A Maven támogatja az almodulos projekteket:

```
<project ...>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.mycompany.app</groupId>
  <artifactId>parent-app</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>pom</packaging>
  <!-- subprojects -->
  <modules>
    <module>first-child-app</module>
    <module>second-child-app</module>
  </modules>
</project>
```


Projekt hierarchiák

- Az almodul projektek szintén hivatkoznak a szülőjükre:

```
<project ...>
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>com.mycompany.app</groupId>
    <artifactId>parent-app</artifactId>
    <version>1.0-SNAPSHOT</version>
  </parent>
  <groupId>com.mycompany.app</groupId>
  <artifactId>first-child-app</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging>
  ...
</project>
```

Pluginek

- A Maven alapfunkciói korlátozottak, de a Maven egy bővíthető (*pluginable*) keretrendszer.
- Számos különböző plugin elérhető
 - például: C++, LaTeX, ant build, javadoc stb.
 - A hivatalos pluginek listája:
<https://maven.apache.org/plugins/>
 - Léteznek harmadik féltől származó pluginek is, és saját pluginok is készíthetők

Plugin példa: Javadoc

```
<project ...>
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-javadoc-plugin</artifactId>
        <version>3.2.0</version>
        <configuration>
          ...
        </configuration>
      </plugin>
    </plugins>
  </build>
  ...
</project>
```

► Dokumentáció generálása: `mvn javadoc:javadoc` vagy `mvn:site`

Függőségek

- Azokat a külső könyvtárakat, amelyeket a projekt használ, függőségeknek nevezzük (*dependencies*).
- A Maven függőségkezelési funkciója biztosítja a könyvtárak automatikus letöltését egy központi tárolóból.

```
<project ...>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

- A GAV egyértelműen azonosítja a szükséges *artifactot*
- A *scope* határozza meg, hogyan használjuk a függőséget

A függőségek *scope*-jai

- A legfontosabb *scope* típusok:
 - **compile:** Alapértelmezett, ha nincs megadva. A fordításhoz szükséges függőségek.
 - **runtime:** Futásidőben szükséges függőségek, de a fordítás során nem szükségesek.
 - **test:** Csak teszteléshez szükséges függőségek, nem részei az éles alkalmazásnak.

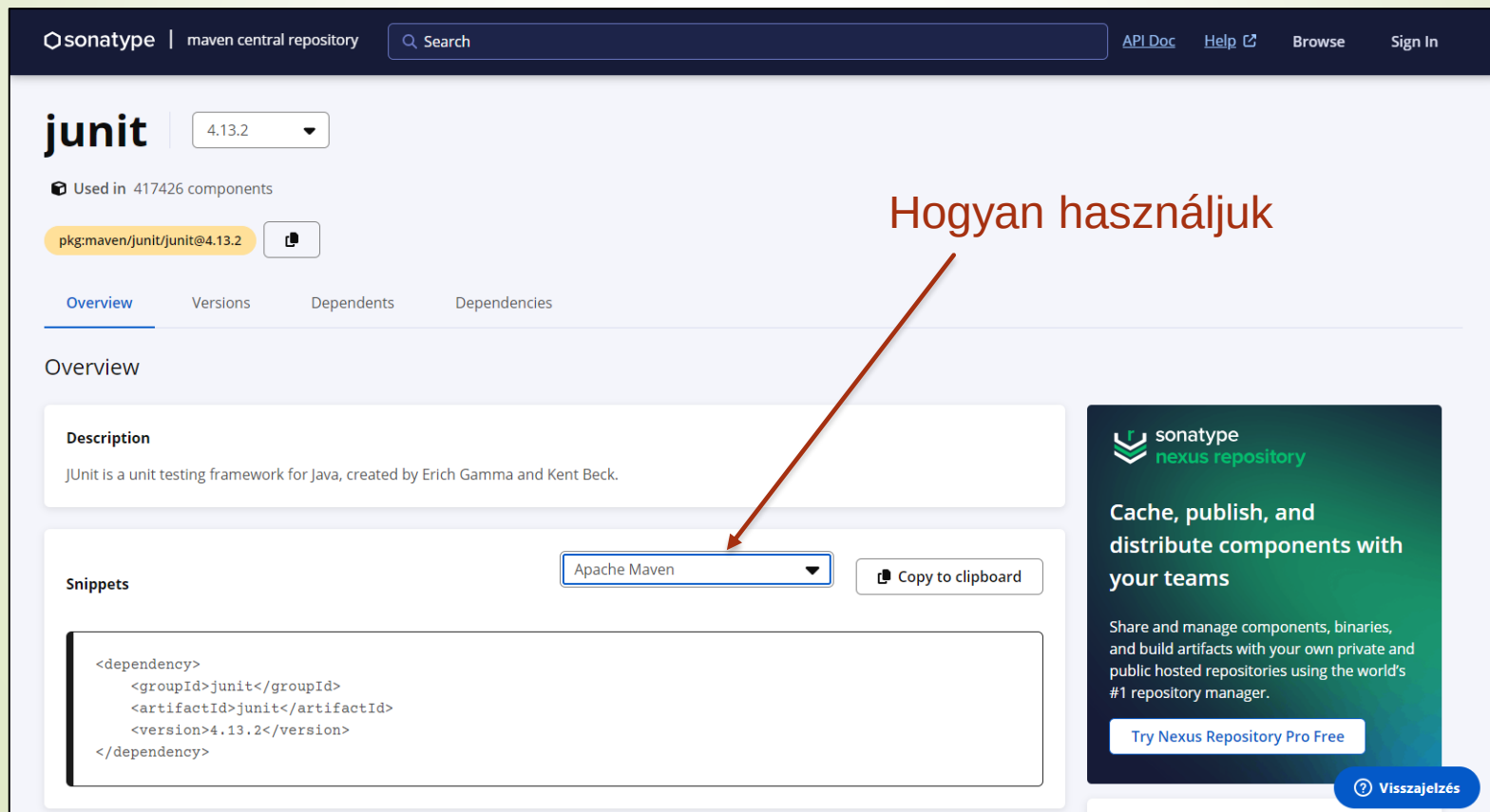
Build rendszerek - Maven

Függőségek keresése

- A Maven Central tárolóban elérhető könyvtárak böngészhetők és kereshetők:

- <https://central.sonatype.com/>

Új alapértelmezett URL és felület:
<https://central.sonatype.com/>



sonatype | maven central repository

Search

API Doc Help Browse Sign In

junit

4.13.2

Used in 417426 components

pkg:maven/junit/junit@4.13.2

Overview Versions Dependents Dependencies

Overview

Description

JUnit is a unit testing framework for Java, created by Erich Gamma and Kent Beck.

Snippets

Apache Maven

Copy to clipboard

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
</dependency>
```

sonatype nexus repository

Cache, publish, and distribute components with your teams

Share and manage components, binaries, and build artifacts with your own private and public hosted repositories using the world's #1 repository manager.

Try Nexus Repository Pro Free

Visszajelzés

Teljes pom.xml fájl a *ThesisGenerator* alkalmazáshoz

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6     <groupId>hu.elte.inf</groupId>
7     <artifactId>thesis-generator</artifactId>
8     <version>1.0-SNAPSHOT</version>
9     <packaging>jar</packaging>
10    <properties>
11        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
12        <maven.compiler.source>15</maven.compiler.source>
13        <maven.compiler.target>15</maven.compiler.target>
14        <maven-jar-plugin.version>3.2.0</maven-jar-plugin.version>
15        <maven-project-info-reports-plugin.version>3.0.0</maven-project-info-reports-plugin.version>
16        <maven-javadoc-plugin.version>3.2.0</maven-javadoc-plugin.version>
17        <junit.version>4.13.2</junit.version>
18    </properties>
19    <build>
20        <sourceDirectory>src</sourceDirectory>
21        <testSourceDirectory>test</testSourceDirectory>
22        <plugins>
23            <plugin>
24                <groupId>org.apache.maven.plugins</groupId>
25                <artifactId>maven-jar-plugin</artifactId>
26                <version>${maven-jar-plugin.version}</version>
27                <configuration>
28                    <archive>
29                        <manifest>
30                            <addClasspath>true</addClasspath>
31                            <mainClass>thesisGenerator.ThesisGenerator</mainClass>
32                        </manifest>
33                    </archive>
34                </configuration>
35            </plugin>
36        </plugins>
37    </build>
```


Teljes pom.xml fájl a *ThesisGenerator* alkalmazáshoz

```
38     <reporting>
39         <plugins>
40             <plugin>
41                 <groupId>org.apache.maven.plugins</groupId>
42                 <artifactId>maven-project-info-reports-plugin</artifactId>
43                 <version>${maven-project-info-reports-plugin.version}</version>
44             </plugin>
45             <plugin>
46                 <groupId>org.apache.maven.plugins</groupId>
47                 <artifactId>maven-javadoc-plugin</artifactId>
48                 <version>${maven-javadoc-plugin.version}</version>
49                 <configuration>
50                     <doctitle>My API for ${project.name} ${project.version}</doctitle>
51                     <windowtitle>My API for ${project.name} ${project.version}</windowtitle>
52                     <show>public</show>
53                 </configuration>
54             </plugin>
55         </plugins>
56     </reporting>
57     <dependencies>
58         <dependency>
59             <groupId>junit</groupId>
60             <artifactId>junit</artifactId>
61             <version>${junit.version}</version>
62             <scope>test</scope>
63         </dependency>
64     </dependencies>
65 </project>
```

Build rendszerek - Gradle



Funkciók

- Egyre népszerűbb build automatizációs rendszer [1] [2]
- Célja az Ant és a Maven legjobb koncepcióinak egyesítése
- Támogatja az inkrementális buildeket
 - Jelentős teljesítménynövekedést nyújt nagyobb vállalati projektek esetén
- Konfiguráció XML helyett Groovy vagy Kotlin-alapú domain specifikus nyelvvel (DSL)
- Hivatalos weboldal: <https://gradle.org/>
- Oktatóanyag:
https://docs.gradle.org/current/userguide/getting_started_eng.html

[1] <https://www.baeldung.com/java-in-2019>

[2] <https://www.jetbrains.com/lp/devecosystem-2023/java/>

build.gradle

- A Gradle-ben a build rendszer a *build.gradle* fájlban van definiálva Groovy nyelven.
 - Kotlin használata esetén: *build.gradle.kts*
 - Alapértelmezett használathoz a konvenció szerint elnevezés használatát kövessük.
- Hogyan hozzunk létre *build.gradle* fájlt?
 1. Írhatjuk kézzel a nulláról
 2. A `gradle init` paranccsal automatikusan
 3. Használhatjuk preferált IDE-nket a generálására

Feladatok (tasks)

- A Gradle-ben a buildek egy vagy több projektből állnak, és minden projekt egy vagy több feladatból (*task*) áll.
 - Egy *task* egyetlen műveletet végez, pl. osztályok fordítása, archívumok létrehozása vagy publikálása.
 - Egyszerű *task* definíció:

```
task hello {  
    doLast {  
        println "Hello World"  
    }  
}
```

Viselkedés hozzáadása meglévő taskokhoz

- Meglévő taskok kiegészíthetők további műveletekkel, prefixálással vagy suffixálással.

```
task hello {  
    doLast {  
        println "Hello World"  
    }  
}  
  
hello.doFirst {  
    println "First Line"  
}  
  
hello.doLast {  
    println "Last Line"  
}
```

Task függőségek

- Függőségi kapcsolat definiálható taskok között a *dependsOn* argumentummal:

```
task init {  
    doLast {  
        ...  
    }  
}  
  
task compile(dependsOn: init) {  
    doLast {  
        ...  
    }  
}
```

Pluginek

- A meglévő taskok listázhatók a `gradle tasks` paranccsal.
- A build rendszer konfigurációjának egyszerűsítésére a Gradle számos beépített plugint kínál, amelyek előre definiált taskokat tartalmaznak.
 - A java plugin használatával elérhetővé válnak taskok általános Java-alapú alkalmazások fordítására, tesztelésére, csomagolására.

```
plugins {  
    id "java"  
}
```

- Néhány parancs: `gradle assemble`, `gradle build`, `gradle jar`, `gradle test`, `gradle clean`

Pluginek

- Az *application* plugin (amely a *java* plugint is magába foglalja) lehetővé teszi futtatható build létrehozását.

```
plugins {  
    id "application"  
}
```

- A manifest belépési pontját még meg kell adni:

```
application {  
    mainClass = "project.MainClass"  
}
```

Függőségek

- A Gradle rugalmas függőségkezelő rendszert támogat.
- Sok konvenciót és megoldást átvett a Maven-ből, például a Maven Central használatának lehetőségét a függőségek forrásaként.

```
repositories {  
    mavenCentral()  
}
```

- A függőségeket különböző konfigurációkba csoportosítjuk, pl. `implementation`, `testImplementation` vagy `runtimeOnly`, amelyek egymást is bővíthetik.

```
dependencies {  
    testImplementation "junit:junit:4.13.2"  
}
```

Teljes build.gradle fájl a *ThesisGenerator* alkalmazáshoz

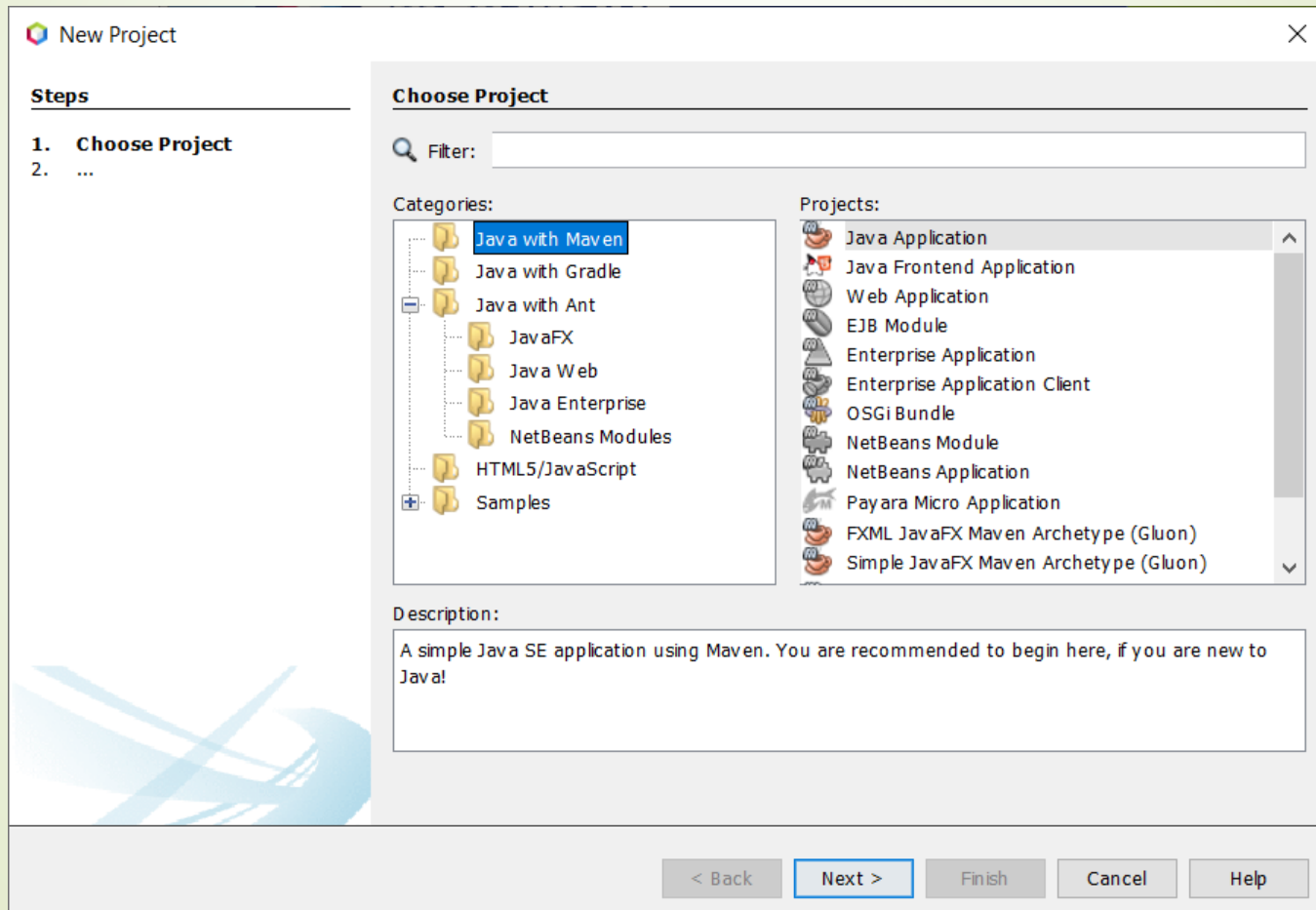
```
1. plugins {  
2.     // Apply the application plugin. Implies the java plugin.  
3.     id 'application'  
4. }  
5. application {  
6.     // Define the main class for the application.  
7.     mainClass = 'thesisGenerator.ThesisGenerator'  
8. }  
9. repositories {  
10.    // Use the Maven Central Repository for dependencies.  
11.    mavenCentral()  
12. }  
13. dependencies {  
14.    // Use JUnit test framework.  
15.    testImplementation 'junit:junit:4.13.2'  
16. }  
17. // Set source directories. Advised to use the conventional directory layout in general.  
18. sourceSets {  
19.     main.java.srcDirs = ['src']  
20.     test.java.srcDirs = ['test']  
21. }  
22. // Set build directory.  
23. project.buildDir = 'build-gradle'  
24.
```

Gradle Wrapper

- Projekt létrehozásakor hozzáadható a *Gradle Wrapper* (gradlew). (Vagy később a `gradle wrapper` paranccsal.)
 - A Gradle Wrapper egy script, amely egy adott verziójú Gradle-t indít el, szükség esetén letölti azt.
- A Gradle build-ek futtatásának ajánlott módja a Gradle Wrapper használata. Előnyei:
 - A projekt létrehozása után nem kell manuálisan Gradle-t telepíteni a fejlesztőkörnyezetbe.
 - A projekt fejlesztői (és CI rendszerek) között rögzített Gradle verziót biztosít.
 - A projekt egyszerűen frissíthető új Gradle verzióra a Wrapper definíció módosításával.
- Verziókezelés: a `gradlew` Bash és Batch script, valamint a `gradle` könyvtár legyenek verziókezelve, a letöltött binárisokat tartalmazó `.gradle` könyvtár viszont ne.

NetBeans

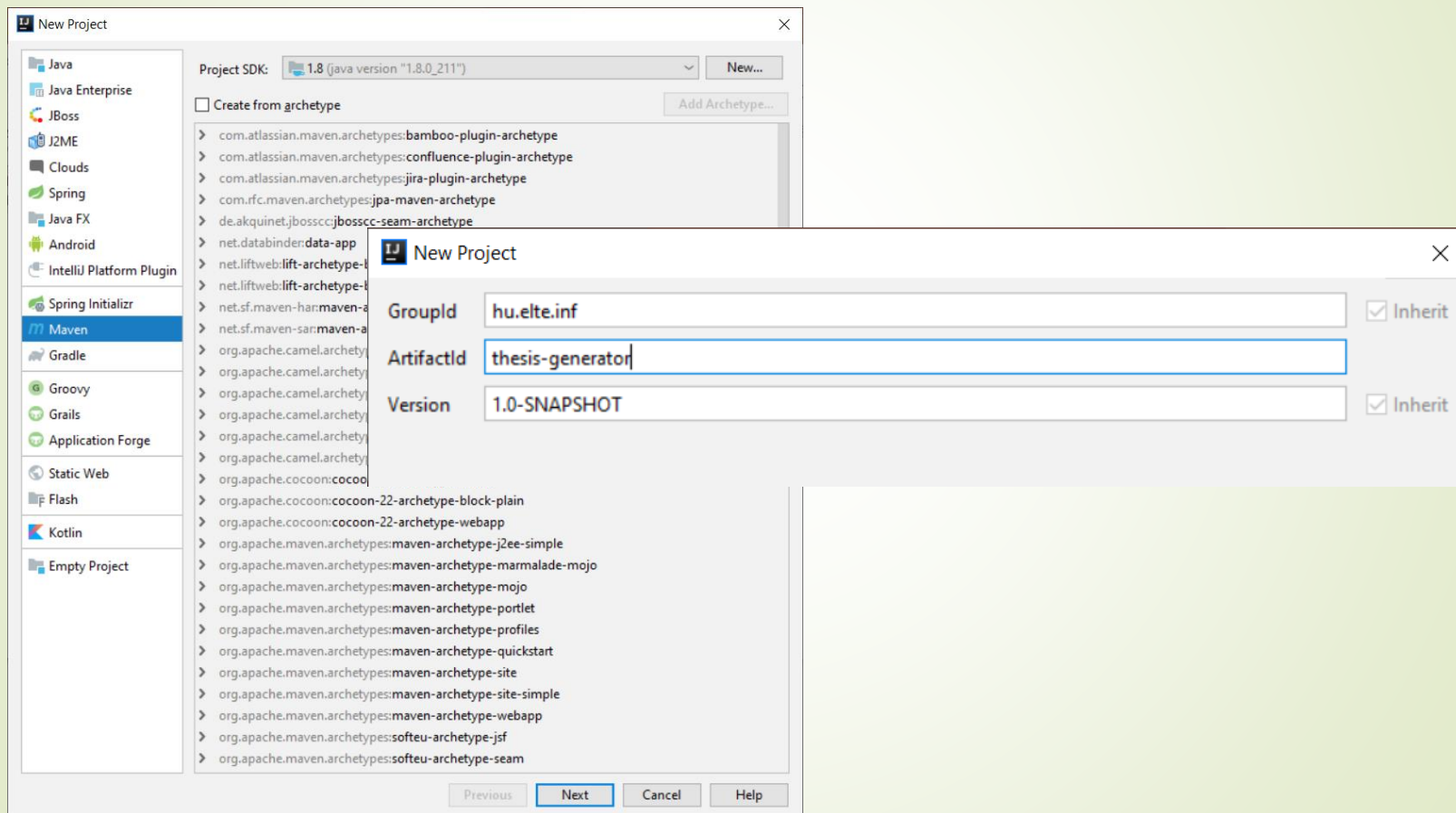
- A NetBeans támogatja az Ant, Maven és Gradle projekteket is.



Build rendszerek – IDE integráció

IntelliJ

- Az IntelliJ IDEA szoros integrációt kínál Maven és Gradle projektekhez.



- *Ant build fájl generálása is támogatott: Build -> Generate Ant Build*